

Optimization of Gradient Coils in MRI

Daniel Abraham, Munir Bshara

March 20, 2026

1 Introduction

Magnetic Resonance Imaging (MRI) is an essential tool of modern medical diagnosis, but its clinical utility is constrained by inherently slow image encoding. This limitation reduces clinical throughput, restricts the observation of fast spatio-temporal physiological phenomena, and negatively impacts patient comfort. Image encoding speed is governed by the ability of gradient coils to generate strong, spatially linear magnetic fields over the region of interest (ROI). In conventional MRI systems, these gradient coils are located along the outer bore to support general-purpose imaging, but this design incurs several penalties for fast imaging. Large-radius gradient coils require substantially higher power, induce strong eddy currents in surrounding conductive structures, and generate stray electric fields that can trigger peripheral nerve stimulation (PNS).

Prior works using high frequency insert gradient axes [1, 2, 3] have a few limitations. First, insert gradients generally offer limited spatial coverage and gradient strength compared with the primary whole-body gradient system, restricting their use to specialized applications or localized imaging. Second, they require substantial additional hardware and system-level integration, increasing cost and implementation complexity. Third, since they augment rather than replace the conventional gradient axes, they only partially mitigate acoustic noise and do not provide a general solution for quiet high-performance imaging.

To address these limitations, we propose anatomically conforming “wearable” gradient coils positioned close to the subject. By tailoring the gradient geometry to the anatomy and reducing coil-to-tissue distance, this approach enables faster imaging with lower power consumption, reduced eddy currents, and improved PNS performance. However, arbitrary current waveforms are not safe with a wearable gradient coil, due to the large accumulated Lorentz forces acting so close to the subject when using low-frequency waveforms. To mitigate this problem, we use our wearable gradient coils as an extra encoding axis driven at a high frequency ($> 10\text{kHz}$), drastically reduces vibrations and strong accumulated Lorentz forces. In this regime, the wearable insert gradient is responsible for fast switching waveforms, while the traditional built in gradient coils are responsible for slow switching waveforms. The extra gradient axis is then used with a wave-encoding [4] strategy to achieve improved parallel imaging performance.

Our contributions are

1. Development of a surface optimized gradient coil design algorithm
2. Construction of an efficient single frequency power amplifier
3. A plug and play calibration technique to characterize system imperfections
4. Demonstration of an ultra-high spatio-temporal resolution EPI scan

2 Theory: Gradient Coil Optimization

In this analysis, we consider two approaches to designing a wearable gradient coil. The first approach is to optimize the currents of N discrete loop coils distributed around the head, forming a matrix gradient (Fig. 1, left). Matrix gradients are easy to assemble and highly flexible, enabling both linear and nonlinear fields, but are relatively inefficient and require multiple independent gradient power amplifiers (GPAs). The second approach is to instead optimize a smooth scalar stream function over an anatomically aware surface

(Fig. 1, right). This yields highly efficient, arbitrary-axis gradients, but translating the stream function into manufacturable winding patterns is non-trivial. Moreover, each stream-function optimization targets only a single gradient axis per surface.

For both techniques, we will discuss our approach to perform joint optimization of current densities and surface geometry parameters.

2.1 Design Variables

For both approaches to gradient design (Fig. 1), two sets of variables are used. The first set $\mathbf{x} \in \mathbb{R}^N$ are convex design variables, and the second set $\boldsymbol{\theta} \in \mathbb{R}^P$ are surface geometry variables. As suggested by the names, optimizing the first set of variables results in a convex optimization problem, while jointly optimizing surface geometry variables typically results in a non-convex optimization problem.

2.1.1 Matrix Gradient

This approach uses N coil elements to construct a matrix gradient array. An example setup is shown in Figure 1 with $N = 5$ coils. We use the variable x_n to denote the ampere-turn value for the n^{th} coil element, and parametrize the N -coil element design through the vector $\mathbf{x} \in \mathbb{R}^N$. The surface geometry parameters $\boldsymbol{\theta}$ include shape parameters (radii for circular loops), positions, and orientations per coil element. A technical description is available in Appendix 6.1 and a detailed computational analysis can be found in Appendix 6.3.

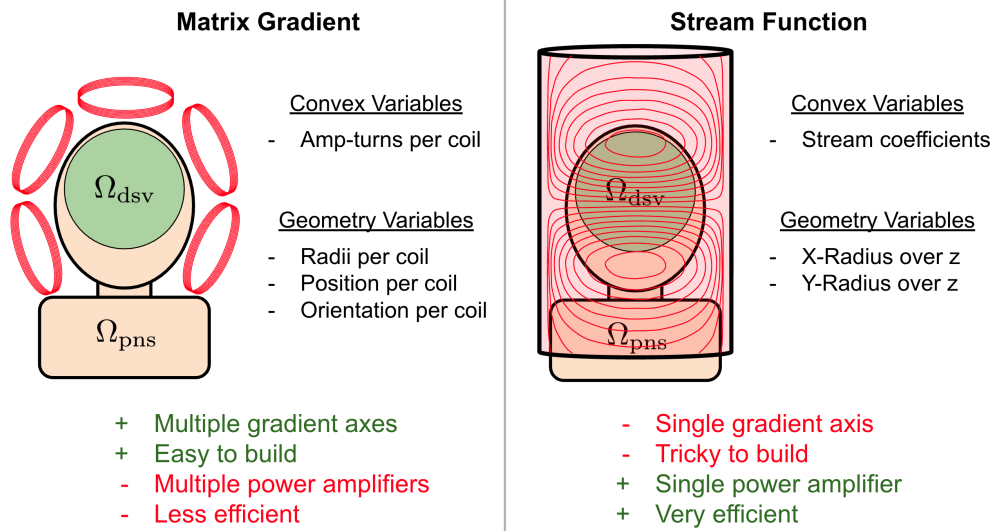


Figure 1: Matrix gradient coil setup. The objective is to generate a strong magnetic field gradient in the green design spherical volume (DSV) region, while keeping the electric fields minimized in the orange regions which are sensitive to peripheral nerve stimulation (PNS).

2.1.2 Stream Function

A technical description of the stream function methodology is provided in Appendix 6.2. The convex design variable $\mathbf{x}$ for the stream function design are the stream function basis coefficients. The bases we use are a combination of Chebyshev polynomials and Fourier bases. The surface geometry parameters $\boldsymbol{\theta}$ contain parameters describing how a cylinder's major and minor radii change as a function of z -position. Figure 1 (right) shows an example y -gradient pattern achieved using the stream function method.

2.2 Gradient Coil Design as Convex Optimization

Several useful quantities can be determined through linear or quadratic functions of the convex design variable $\mathbf{x}$:

$$\begin{aligned}\mathbf{G}(\mathbf{r})\mathbf{x} &\rightarrow \text{gradient field at spatial coordinate } \mathbf{r} \\ \mathbf{E}(\mathbf{r})\mathbf{x} &\rightarrow \text{electric field at spatial coordinate } \mathbf{r} \\ \mathbf{x}^T \mathbf{L} \mathbf{x} &\rightarrow \text{magnetic energy of the coil,}\end{aligned}$$

where $\mathbf{G}(\mathbf{r}) \in \mathbb{R}^{3 \times N}$ is the gradient field matrix describing the spatial derivative of the magnetic field z -component along the three coordinate axes, $\mathbf{E}(\mathbf{r}) \in \mathbb{R}^{3 \times N}$ is the electric field matrix, and $\mathbf{L} \in \mathbb{R}^{N \times N}$ is the coil's magnetic energy matrix. All the aforementioned matrices are constructed using knowledge of the surface geometry parameters $\boldsymbol{\theta}$. Our objective is to design a gradient coil that meets the following criteria

- 1.) The gradient strength is at least $G_{\min}$ over all coordinates $\mathbf{r} \in \Omega_{\text{dsv}}$.
- 2.) The induced electric field magnitudes are no larger than $E_{\max}$ over all coordinates $\mathbf{r} \in \Omega_{\text{pns}}$.
- 3.) The coil(s) magnetic energy is no larger than $L_{\max}$.

Here we use Ω_{dsv} to denote the set of all coordinates $\mathbf{r}$ where we want to achieve a strong gradient field, and Ω_{pns} as the set of all coordinates $\mathbf{r}$ where we want to reduce electric fields to mitigate PNS. The design objectives above can be formulated as a convex optimization problem

$$\begin{aligned}\min_{\mathbf{x}, G_{\min}, L_{\max}, E_{\max}} \quad & -\lambda_G G_{\min} + \lambda_L L_{\max} + \lambda_E E_{\max} \\ \text{s.t.} \quad & \mathbf{e}_d^T \mathbf{G}(\mathbf{r})\mathbf{x} \geq G_{\min}, \forall \mathbf{r} \in \Omega_{\text{dsv}} \\ & \|\mathbf{E}(\mathbf{r})\mathbf{x}\|_2 \leq E_{\max}, \forall \mathbf{r} \in \Omega_{\text{pns}} \\ & \mathbf{x}^T \mathbf{L} \mathbf{x} \leq L_{\max} \\ & \mathbf{A} \mathbf{x} = \mathbf{b} \\ & \mathbf{C} \mathbf{x} \leq \mathbf{d},\end{aligned} \tag{1}$$

where $\mathbf{e}_d \in \mathbb{R}^3$ is a one-hot vector selecting the gradient axis to optimize over and $\lambda_G, \lambda_L, \lambda_E$ are the hyperparameters used to emphasize different design criteria. We note that it is also possible, to use $G_{\min}, L_{\max}, E_{\max}$ directly as hyperparameters instead of $\lambda_G, \lambda_L, \lambda_E$. The matrices $\mathbf{A} \in \mathbb{R}^{M_A \times N}$, $\mathbf{C} \in \mathbb{R}^{M_C \times N}$ and vectors $\mathbf{b} \in \mathbb{R}^{M_A}$, $\mathbf{d} \in \mathbb{R}^{M_C}$ are used to enforce optional linear equality and inequality constraints if desired. We solve the above optimization problem using ADMM [5]. The detailed set of ADMM update equations can be found in Appendix 6.4.

2.3 Optimization of Geometry Variables

In order to improve the design objective tradeoffs, we propose a joint optimization of convex design vector $\mathbf{x}$ along with coil geometry parameters $\boldsymbol{\theta}$. The relevant field and inductance matrices are now functions of the geometry parameters, yielding the new optimization problem

$$\begin{aligned}
& \min_{\boldsymbol{\theta}, \mathbf{x}, G_{\min}, L_{\max}, E_{\max}} && -\lambda_G G_{\min} + \lambda_L L_{\max} + \lambda_E E_{\max} \\
& \text{s.t.} && \\
& \mathbf{e}_d^T \mathbf{G}_{\boldsymbol{\theta}}(\mathbf{r}) \mathbf{x} \geq G_{\min}, \forall \mathbf{r} \in \Omega_{\text{dsv}} && \\
& \|\mathbf{E}_{\boldsymbol{\theta}}(\mathbf{r}) \mathbf{x}\|_2 \leq E_{\max}, \forall \mathbf{r} \in \Omega_{\text{pns}} && (2) \\
& \mathbf{x}^T \mathbf{L}_{\boldsymbol{\theta}} \mathbf{x} \leq L_{\max} && \\
& \mathbf{A}_{\boldsymbol{\theta}} \mathbf{x} = \mathbf{b}_{\boldsymbol{\theta}} && \\
& \mathbf{C}_{\boldsymbol{\theta}} \mathbf{x} \leq \mathbf{d}_{\boldsymbol{\theta}}. &&
\end{aligned}$$

The above optimization problem is no longer convex. Our general approach to solving this is to unroll the ADMM iterations for a fixed $\boldsymbol{\theta}$, and wrap the entire unrolled process in a stochastic gradient descent (SGD) algorithm [6] to optimize $\boldsymbol{\theta}$. This dramatically increases the amount of computation needed to solve the optimization problem, mainly due to the new requirement of forming the geometric dependent physics matrices $\mathbf{G}_{\boldsymbol{\theta}}$, $\mathbf{E}_{\boldsymbol{\theta}}$, $\mathbf{L}_{\boldsymbol{\theta}}$, and optionally $\mathbf{A}_{\boldsymbol{\theta}}$, $\mathbf{b}_{\boldsymbol{\theta}}$, $\mathbf{C}_{\boldsymbol{\theta}}$, $\mathbf{d}_{\boldsymbol{\theta}}$, after every SGD step.

3 Results

Before interpreting the optimization output, it was necessary to verify that the underlying field, electric-field, and inductance operators were computed with sufficient numerical accuracy. To that end, we first studied quadrature convergence for the matrices entering (1), namely $\mathbf{G}(\mathbf{r})$, $\mathbf{E}(\mathbf{r})$, and $\mathbf{L}$. For the straight-wire and circular cases, closed-form or semi-analytic references are available, while for the elliptical case no comparable closed-form reference exists. We therefore define the golden reference for the elliptical-loop experiments using a much finer midpoint discretization, and compare coarser midpoint and Gauss–Legendre rules against that baseline. This gives a consistent numerical target without introducing a second modeling approximation as seen in Figure 2.

These convergence studies show that both midpoint and Gauss–Legendre quadrature recover the correct asymptotic behavior, but with different accuracy–cost tradeoffs. Midpoint rules are simple and robust, while Gauss–Legendre rules achieve higher accuracy per sample when the kernel is sufficiently smooth. For the matrix-coil formulation, this is especially useful because each loop integral can be evaluated independently and efficiently; for the stream-function formulation, the same idea extends to tensor-product quadrature after the surface change of variables in Appendix 6.2.6.

A second important numerical observation is that the inductance model, although approximate, is sufficiently accurate for optimization and convergence studies. The inductance calculation is not exact because the magnetic-energy integral contains a near-singular, and in the coincident limit singular, kernel of the form $1/\|\mathbf{r} - \mathbf{r}'\|$, so any finite quadrature rule necessarily approximates this interaction rather than resolving it analytically. In particular, the inductance term is the dominant expensive quantity because it is defined by a double interaction integral, but the approximation used here remains stable and predictive enough to preserve the qualitative ordering of designs and the observed convergence trends. Thus, while the inductance calculation is not exact, it is accurate enough for strong convergence evidence and for comparing optimization outcomes under a fixed modeling pipeline.

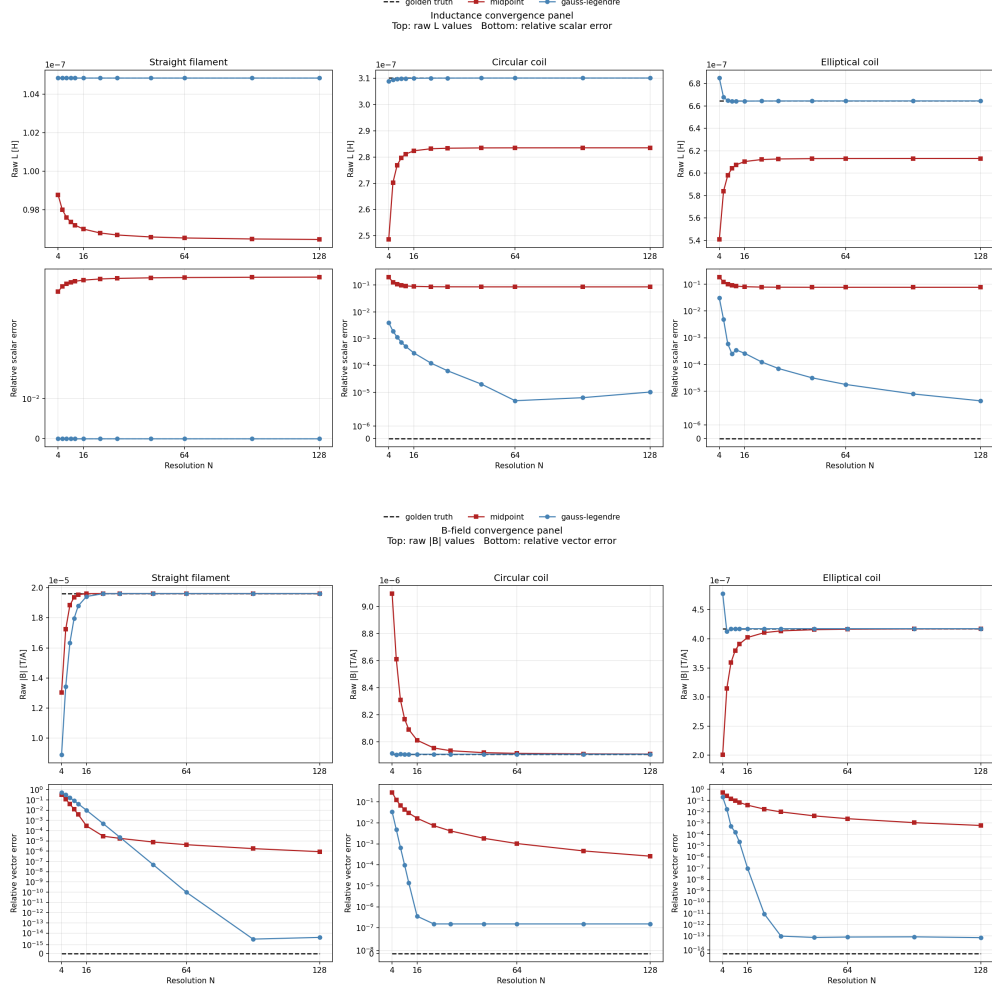


Figure 2: Quadrature comparison on 3 geometries: straight filament(left), circular loop(middle), elliptical(right).

Using these results in conjunction with the optimization process detailed by Equation 2 and Equation 1, one can with 500 iterations of ADMM obtain reasonable convex results. Figure 3 is a first proof of functionality and correctness by working with a simple cylindrical surface. One can notice that the z-axis gradient is better by a substantial margin when using the matrix coil method. This comes as a surprise as the coils designed by the surface optimizer should have magnetic field all point in that same direction thus contributing to this gradient. For the x-axis gradient on the bottom there are no surprising results and both optimization methods are equally poor due to the difficulty of this problem. Note that due to the symmetries in the problem, evaluating x-axis gradient and y-axis gradient separately may be omitted and all results from one can be transposed onto the other.

The second surface studied we name as an ergonomic helmet type surface. Unfortunately this surface

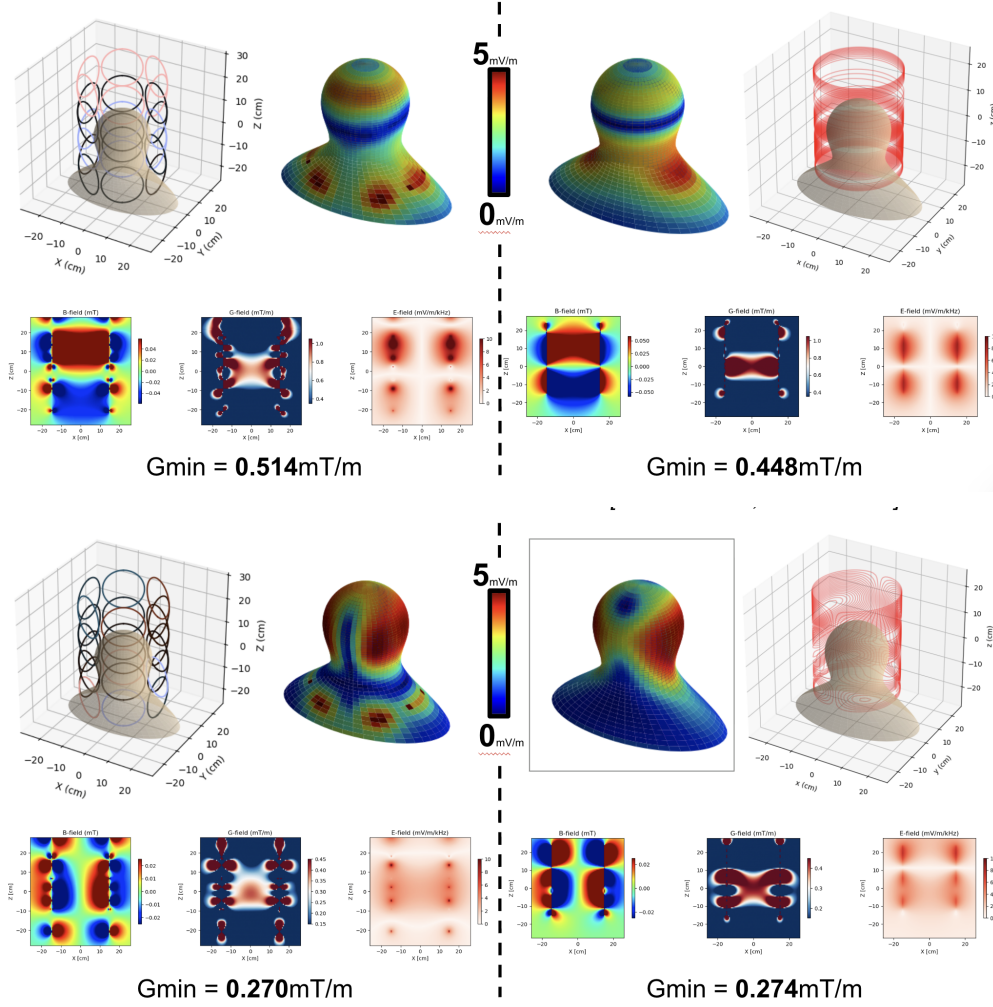


Figure 3: Optimization of z-axis gradient(top) and x-axis gradient(bot) over cylindrical surface with both matrix gradient(left) and stream function(right).

seems to lead to equal if not worse results as the naive cylindrical construction. Intuitively, this is expected as you are constraining loops in directions in which their interactions would directly oppose each other more easily.

Lastly, we encountered some issues with the full non-convex surface optimization and so we provide some quantifiable evidence on why different types of surfaces should be looked at. In Figure 5, we used a very simply transformed helmet shape which tapers towards the top. One can notice that with this slight change to the surface we get a significant improvement in gradient measurement in both directions tested.

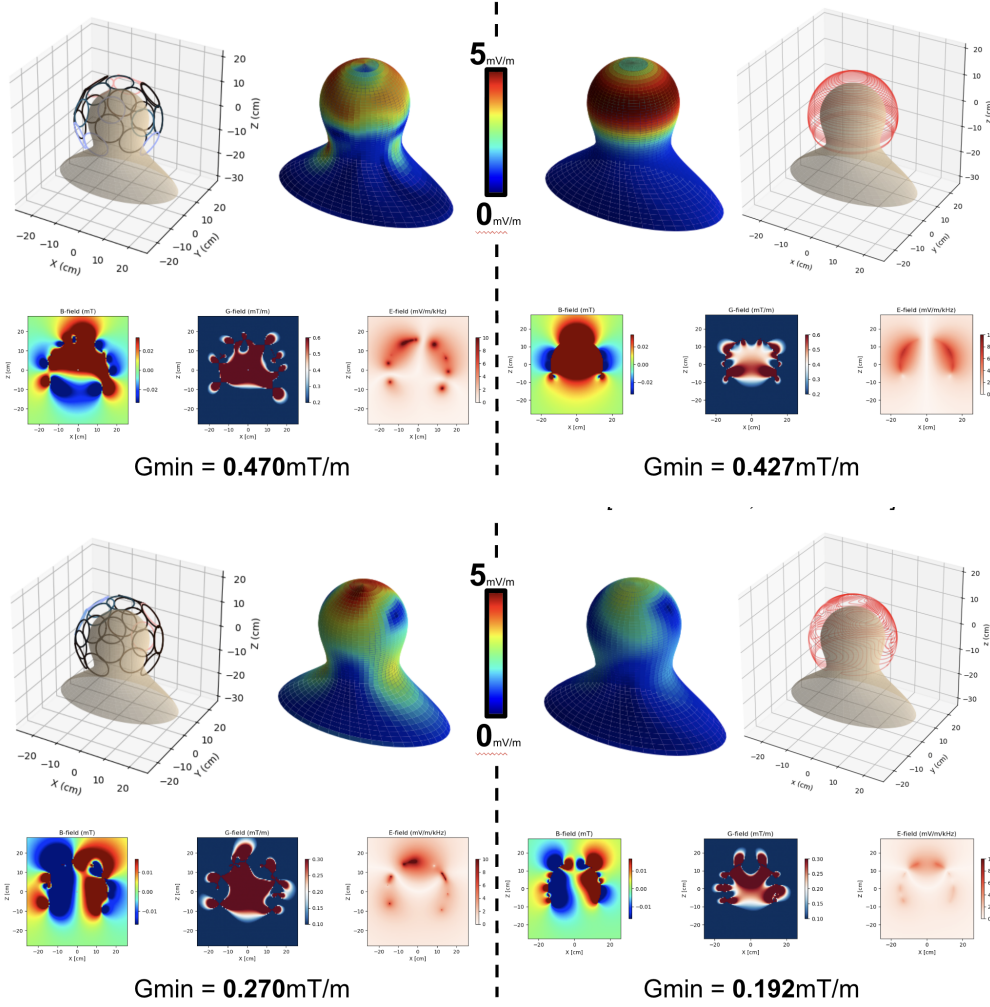


Figure 4: Optimization of z-axis gradient(top) and x-axis gradient(bot) over helmet surface with both matrix gradient(left) and stream function(right).

4 Conclusions

This project developed a common optimization framework for two wearable MRI gradient-coil design strategies: matrix gradients built from independently driven loop elements, and stream-function coils built from a continuous surface-current representation. Although the physical realizations differ, both approaches admit the same basic structure once the design is expanded in a finite basis and the relevant gradient-field, electric-field, and magnetic-energy quantities are assembled into linear and quadratic operators. This common viewpoint is useful because it places coil design, safety constraints, and geometry variation into one computational language.

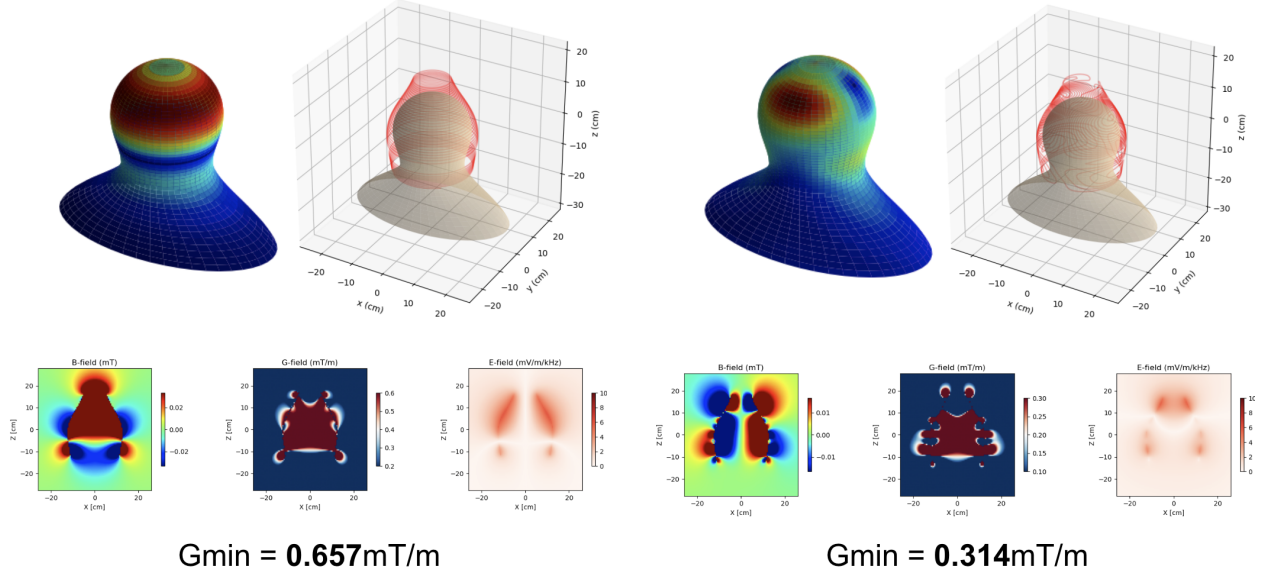


Figure 5: Optimization using stream function approach of z-axis gradient(left) and x-axis gradient(right) over manually optimized surface.

The results show that neither design family is uniformly best. Matrix gradients offer flexibility and simpler element-wise control, while stream-function methods provide a natural route to connected, anatomically conforming winding patterns. Which approach is preferable depends on the desired gradient axis, the chosen surface, the allowable inductive load, and how accurately peripheral nerve stimulation is modeled. In particular, the surface-charge correction to the electric field is not a minor detail: it changes the effective PNS constraint and therefore changes which designs appear favorable.

Another reason why quantifying the better approach comes with from the practical difficulty in the optimization is that the gradient-field, electric-field, and inductance terms naturally live on very different numerical scales. The gradient objective is measured through spatial field derivatives, the electric-field constraint is measured pointwise over the PNS surface, and the inductance term arises from a quadratic energy functional, so their raw magnitudes can differ substantially even for the same design. As a result, the constants λ_G , λ_E , and λ_L in (1) do more than express preference: they also compensate for scale mismatch between competing physics terms. If these weights are not chosen carefully, the optimizer may appear to favor one metric simply because it is numerically larger or smaller, rather than because it is physically more important. Finding a balanced set of constants is therefore essential to ensure that improvements in gradient strength, electric-field suppression, and inductive cost are all represented comparably in the optimization.

Regarding accuracy of our approach, we note that under the present linear field approximations, some designs may appear better or worse in optimization than they would in practice, since singular or near-singular interactions such as self-inductance and cross-inductance between spatially overlapping loops is only captured approximately. Any of the matrix optimization figures, is a natural place to illustrate these overlapping current paths and their associated interactions. Quantifying this effect remains future work, and should be addressed before drawing strong engineering conclusions or committing substantial design effort to a particular geometry.

Another practical physics consideration is the effect of RF operation on inter-coil interactions. Even if the low-frequency gradient model is satisfactory for the present optimization, RF coupling and frequency-dependent electromagnetic interactions between nearby conductors may alter the effective behavior of the coil assembly in practice, and should therefore be accounted for in a more complete hardware-aware design pipeline.

The most promising direction for future work is shape optimization. Determining how to optimize the geometry while still guaranteeing a realizable and manufacturable coil remains an open problem. In that setting, more structured parameterizations, such as elliptical coils, squircles, or other constrained shape families, may provide a better balance between geometric flexibility and physical realizability.

Overall, substantial work remains before this pipeline can be considered fully rigorous. In addition to improving the electromagnetic and geometric modeling, it may also be valuable to refine the inverse-problem metrics so that consistency and variability are incorporated more explicitly, particularly to account for RF-related inaccuracies. At the same time, RF behavior in MRI is inherently sensitive, so such refinements should be viewed as improving robustness rather than eliminating all mismatch.

5 Use of AI-Assisted Tooling

AI-based tools were used in a limited and assistive role during this project. In particular, they were used as debugging aids during implementation and, in some cases, to help draft or auto-generate test scaffolding that was later reviewed and refined. These tools were not used as autonomous agents that executed design decisions or modified the project without oversight.

All AI-assisted outputs were checked by humans before use. Human oversight was maintained in both debugging and test generation, and no AI-generated code, derivation, or modification was automatically executed without manual inspection. The final responsibility for correctness, implementation decisions, and interpretation of results remained entirely with the human authors.

References

- [1] E. Versteeg, D. W. Klomp, and J. C. Siero, “A silent gradient axis for soundless spatial encoding to enable fast and quiet brain imaging,” *Magnetic resonance in medicine*, vol. 87, no. 2, pp. 1062–1073, 2022.
- [2] N. L. Dispenza, S. Littin, M. Zaitsev, R. T. Constable, and G. Galiana, “Clinical potential of a new approach to mri acceleration,” *Scientific reports*, vol. 9, no. 1, p. 1912, 2019.
- [3] K. Scheffler, A. Loktyushin, J. Bause, A. Aghaeifar, T. Steffen, and B. Schölkopf, “Spread-spectrum magnetic resonance imaging,” *Magnetic resonance in medicine*, vol. 82, no. 3, pp. 877–885, 2019.
- [4] B. Bilgic, B. A. Gagoski, S. F. Cauley, A. P. Fan, J. R. Polimeni, P. E. Grant, L. L. Wald, and K. Setsompop, “Wave-caipi for highly accelerated 3d imaging,” *Magnetic resonance in medicine*, vol. 73, no. 6, pp. 2152–2162, 2015.
- [5] S. Boyd, N. Parikh, E. Chu, B. Peleato, J. Eckstein, *et al.*, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends® in Machine learning*, vol. 3, no. 1, pp. 1–122, 2011.

- [6] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter, “Differentiable convex optimization layers,” *Advances in neural information processing systems*, vol. 32, 2019.
- [7] J. C. Simpson, J. E. Lane, C. D. Immer, and R. C. Youngquist, “Simple analytic expressions for the magnetic field of a circular current loop,” tech. rep., 2001.
- [8] T. J. Rivlin, *Chebyshev polynomials*. Courier Dover Publications, 2020.
- [9] P. B. Roemer and B. K. Rutt, “Minimum electric-field gradient coil design: Theoretical limits and practical guidelines,” *Magnetic resonance in medicine*, vol. 86, no. 1, pp. 569–580, 2021.
- [10] P. B. Roemer, T. Wade, A. Alejski, C. A. McKenzie, and B. K. Rutt, “Electric field calculation and peripheral nerve stimulation prediction for head and body gradient coils,” *Magnetic resonance in medicine*, vol. 86, no. 4, pp. 2301–2315, 2021.
- [11] G. H. Golub and J. H. Welsch, “Calculation of gauss quadrature rules,” *Mathematics of Computation*, vol. 23, no. 106, pp. 221–230, 1969.

6 Appendix

6.1 Matrix Coil Methodology

The matrix coil method uses several instances of a simple coil element to construct a gradient field. Below we discuss details on how we parameterize the matrix coil design.

6.1.1 Defining the Current and Geometry parameters

We consider the n^{th} coil element, which is a circular loop coil with radius a_n and T_n turns. The center of the coil element is located at the coordinate $\mathbf{c}_n \in \mathbb{R}^3$, and has an orientation defined by a normal vector $\hat{\mathbf{n}}_n \in \mathbb{R}^3$, $\|\hat{\mathbf{n}}_n\|_2 = 1$, orthogonal to the circular winding direction. If we have N total coil elements, then the surface geometry parameters are

$$\boldsymbol{\theta} = \left(a_n, \mathbf{c}_n, \hat{\mathbf{n}}_n \right)_{n=1}^N.$$

The design variable is the product of the current I_n and number of turns T_n for each coil element

$$\mathbf{x} = [I_1 T_1 \quad \cdots \quad I_N T_N].$$

6.1.2 Power, Inductance and Rise Time Constraints

Since the matrix gradient coil is driven by N independent gradient power amplifiers, it is important to keep the per-channel power and inductance limited. Furthermore, we’d like the peak current rise time to be short, which is mainly dictated by the ratio of the inductance to the resistance. More formally our constraints for the n^{th} coil are

$$P_n \leq P_{\max} \tag{3}$$

$$L_n \leq L_{\max} \tag{4}$$

$$\tau_n \leq \tau_{\max}, \tag{5}$$

where P_n, L_n, τ_n is the power, inductance, and rise time for the n^{th} coil respectively, each with corresponding max values. The power is given by $P_n = I_n^2 R_n = \frac{x_n^2}{T_n^2} 2\pi \rho a_n T_n$, where ρ is the resistance per meter of the wire. Note that ρ is frequency dependent (skin effect), and hence the value of ρ for the frequency of interest should be used. The inductance is given by $L_n = T_n^2 L_{\text{circle}}(a_n)$, where $L_{\text{circle}}(a_n)$ is the inductance of a single circular loop with radius a_n . And finally the rise time is given by $\tau_n = \frac{T_n^2 L_{\text{circle}}(a_n)}{2\pi \rho a_n T_n}$. Using these relationships yield inequalities in T_n

$$\frac{2\pi \rho a_n x_n^2}{P_{\text{max}}} \leq T_n \quad (6)$$

$$T_n \leq \sqrt{\frac{L_{\text{max}}}{L_{\text{circle}}(a_n)}} \quad (7)$$

$$T_n \leq \frac{\tau_{\text{max}} 2\pi \rho a_n}{L_{\text{circle}}(a_n)}. \quad (8)$$

Thus, in order for our design variable $\mathbf{x}$ to be realizable with some value of T_n turns, we need the difference between the lower and upper bounds on T_n to be greater than one, and hence we can pick some integer number of turns between the lower and upper bound. This can be realized by enforcing the following inequality on x_n :

$$x_n^2 \leq \underbrace{\frac{P_{\text{max}}}{2\pi \rho a_n} \left(\min \left\{ \sqrt{\frac{L_{\text{max}}}{L_{\text{circle}}(a_n)}}, \frac{\tau_{\text{max}} 2\pi \rho a_n}{L_{\text{circle}}(a_n)} \right\} - 1 \right)}_{U_n}$$

$$\Rightarrow -\sqrt{U_n} \leq x_n \leq \sqrt{U_n}.$$

6.1.3 Computing Field Matrices

One big motivation for using circular coil elements is that the field patterns can be computed semi-analytically, requiring only an elliptical integral lookup table. The expressions for the magnetic field $\mathbf{B}_n(\mathbf{r})$, gradient field $\frac{\partial B_n^{(z)}(\mathbf{r})}{\partial \mathbf{r}}$, magnetic vector potential $\mathbf{A}_n(\mathbf{r})$ can be found in [7].

6.2 Stream Function Methodology

The stream function method optimizes a smooth scalar function over the gradient surface. This scalar function, known as the stream function, is used to the surface current density, which is then used to compute quantities like inductance and induced electric & magnetic fields.

6.2.1 Surface Parameterization

Let us define a gradient coil surface $S \subset \mathbb{R}^3$ parameterized by coordinates $\mathbf{r}(\theta, z)$, where θ, z represent the X-Y plane angular position and Z-positions respectively:

$$\mathbf{r}(\theta, z) = \begin{bmatrix} a(z) \cos(\theta) \\ b(z) \sin(\theta) \\ z \end{bmatrix} \quad (9)$$

The functions $a(z), b(z)$ represent the X and Y radii as a function of Z -position, which results in an elliptical frustum. Note that if $a(z) = b(z) = r$ for some constant r , the resulting surface is a cylinder with radius r . The $a(z), b(z)$ functions allow the cylinder major and minor radii to change as a function of z -position. Thus, the surface geometry parameters are

$$\boldsymbol{\theta} = (a(z), b(z)).$$

When constructing field matrices, it will be useful to have a convenient method of computing the normal vector $\hat{\mathbf{n}}$ over the surface and the area of a surface patch dS as a function of θ, z . The normal vector can be computed via the cross product of the surface tangents

$$\begin{aligned} \frac{\partial \mathbf{r}(\theta, z)}{\partial \theta} &= \begin{bmatrix} -a(z) \sin(\theta) \\ b(z) \cos(\theta) \\ 0 \end{bmatrix}, \quad \frac{\partial \mathbf{r}(\theta, z)}{\partial z} = \begin{bmatrix} a'(z) \cos(\theta) \\ b'(z) \sin(\theta) \\ 1 \end{bmatrix} \\ \mathbf{n}(\theta, z) &= \frac{\partial \mathbf{r}(\theta, z)}{\partial \theta} \times \frac{\partial \mathbf{r}(\theta, z)}{\partial z}, \end{aligned} \tag{10}$$

and the unit normal and surface patch area can be computed using the normal

$$\begin{aligned} dS(\theta, z) &= \|\mathbf{n}(\theta, z)\|_2 d\theta dz \\ \hat{\mathbf{n}}(\theta, z) &= \frac{\mathbf{n}(\theta, z)}{\|\mathbf{n}(\theta, z)\|_2}. \end{aligned} \tag{11}$$

6.2.2 Stream Function Definition

We'd like to optimize the current density $\mathbf{J}(\mathbf{r})$ over the surface S . $\mathbf{J}(\mathbf{r})$ cannot contain sources on the surface, which can be expressed using the following divergence free constraint

$$\nabla_S \cdot \mathbf{J} = 0, \tag{12}$$

where $\nabla_S \cdot \{\}$ is the surface divergence operator. It would be difficult to directly optimize $\mathbf{J}$, since we'd need to iteratively ensure that it is divergence free over the surface. Instead, we can define a scalar function $\phi(\theta, z)$ over the surface S , and relate the current density to $\phi(\theta, z)$ according to

$$\mathbf{J} = \nabla_S \phi \times \hat{\mathbf{n}}, \tag{13}$$

where ∇_S denotes the surface gradient operator, and $\hat{\mathbf{n}}$ is the outward unit normal to the surface. This construction ensures that $\nabla_S \cdot \mathbf{J} = 0$. We refer to $\phi(\theta, z)$ as the stream function. In order to compute the fields produced by the gradient coil, we can first compute the current density from the stream function using (13), and then use integral equations as detailed in Appendix 6.2.6.

6.2.3 Computing the Current Density

To compute the current density using (13), we need to compute $\nabla_S \phi$, which is not trivial and requires carefully understanding what a surface gradient operator does. The surface gradient vector should describe how moving in some direction $d\mathbf{r}$ changes the stream function along the surface:

$$\nabla_S \phi \cdot d\mathbf{r} = d\phi. \tag{14}$$

We can expand the differential terms into

$$\begin{aligned} d\mathbf{r} &= \frac{\partial \mathbf{r}}{\partial \theta} d\theta + \frac{\partial \mathbf{r}}{\partial z} dz \\ d\phi &= \frac{\partial \phi}{\partial \theta} d\theta + \frac{\partial \phi}{\partial z} dz. \end{aligned} \quad (15)$$

Since equation (14) must hold for all $d\theta, dz$, we can generate two sets of equations (one when $d\theta = 0$ and another when $dz = 0$) to solve for $\nabla_S \phi$. Since the surface gradient is defined only along the surface, we can write the surface gradient vector in terms of the surface tangents $\nabla_S \phi = \alpha_\theta \frac{\partial \mathbf{r}}{\partial \theta} + \alpha_z \frac{\partial \mathbf{r}}{\partial z}$. We can now solve for $\boldsymbol{\alpha} = [\alpha_\theta \ \alpha_z]^T$ using (14) and (15)

$$\underbrace{\begin{bmatrix} \frac{\partial \mathbf{r}}{\partial \theta} \cdot \frac{\partial \mathbf{r}}{\partial \theta} & \frac{\partial \mathbf{r}}{\partial \theta} \cdot \frac{\partial \mathbf{r}}{\partial z} \\ \frac{\partial \mathbf{r}}{\partial z} \cdot \frac{\partial \mathbf{r}}{\partial \theta} & \frac{\partial \mathbf{r}}{\partial z} \cdot \frac{\partial \mathbf{r}}{\partial z} \end{bmatrix}}_{\mathbf{G}} \boldsymbol{\alpha} = \begin{bmatrix} \frac{\partial \phi}{\partial \theta} \\ \frac{\partial \phi}{\partial z} \end{bmatrix}, \quad (16)$$

which yields our final expression for $\nabla_S \phi$

$$\nabla_S \phi = \begin{bmatrix} \frac{\partial \mathbf{r}}{\partial \theta} & \frac{\partial \mathbf{r}}{\partial z} \end{bmatrix} \mathbf{G}^{-1} \begin{bmatrix} \frac{\partial \phi}{\partial \theta} \\ \frac{\partial \phi}{\partial z} \end{bmatrix}. \quad (17)$$

6.2.4 Stream function to Manufacturable Winding Patterns

In order to realize a stream function design, current carrying wires are placed on the level sets of the stream function. We denote the k^{th} level set as

$$C_k = \{\mathbf{r}(\theta, z) | \phi(\theta, z) = k\Delta\phi\} \subset \mathbb{R}^3, \quad (18)$$

where $\Delta\phi$ is chosen to be the current through the wire, which follows the path traced by C_k .

If the level sets of ϕ are highly spatially varying or are very closely packed, then the winding patterns may be difficult to realize with real copper wire. In order to solve this, we can constrain the maximum allowable change in ϕ according to

$$\|\nabla_S \phi\|_2 \leq \frac{\Delta\phi}{s_{\min}},$$

where $\Delta\phi$ is the level set spacing (equivalent to the current applied to the gradient coil) and $s_{\min}$ is the minimum tolerable winding spacing.

An important detail when using stream function techniques is make sure that current does not flow out of the surface, otherwise we would need several current sources to power the gradient coil. Thus we can add the following boundary constraint

$$\mathbf{J}(\mathbf{r}_{\text{boundary}}) \cdot \hat{\mathbf{n}}_{\text{out}}(\mathbf{r}_{\text{boundary}}) = 0, \quad (19)$$

where $\mathbf{r}_{\text{boundary}}$ are the surface coordinates of the boundaries of the gradient coil, and $\hat{\mathbf{n}}_{\text{out}}(\mathbf{r}_{\text{boundary}})$ is the outwards facing normal vector with respect to the boundary.

6.2.5 Stream function Basis Expansion

We use the following basis expansion to parameterize the stream function over the surface

$$B_{m,n}(\theta, z) = e^{jm\theta} T_n(\tilde{z}) \quad (20)$$

$$\phi(\theta, z) = \sum_{m,n} B_{m,n}(\theta, z) \phi_{m,n}, \quad (21)$$

where T_n is the n^{th} Chebyshev polynomial [8] and $\tilde{z}$ is a scaled version of z such that $\tilde{z} \in [-1, 1]$. The convex design variable then becomes

$$\mathbf{x} = [\phi_{1,1} \quad \cdots \quad \phi_{M,N}]^T. \quad (22)$$

6.2.6 Computing Field and Inductance Matrices

The general strategy for computing magnetic fields, electric fields, and inductance matrices is to first obtain the current density from a single stream basis function using (17), and then relate the basis current densities to the desired field and inductance quantities using integral equations. This is repeated for each stream basis function.

We first define $\mathbf{J}_n(\mathbf{r})$ as the surface current density produced from the n^{th} stream basis function. The field contributions are defined as $\mathbf{B}_n(\mathbf{r})$ and $\mathbf{A}_n(\mathbf{r})$ for the magnetic field and magnetic vector potential, respectively, from the n^{th} stream basis function. We also define the magnetic energy matrix $W_{m,n}$ describing cross terms for the $m^{\text{th}}, n^{\text{th}}$ stream basis functions. These quantities can be computed using integrals

$$\mathbf{B}_n(\mathbf{r}) = \frac{\mu_0}{4\pi} \int_{S'} \frac{\mathbf{J}_n(\mathbf{r}') \times (\mathbf{r} - \mathbf{r}')}{\|\mathbf{r} - \mathbf{r}'\|^3} dS' \quad (23)$$

$$\begin{aligned} \frac{\partial B_n^{(z)}(\mathbf{r})}{\partial \mathbf{r}} &= \frac{\mu_0}{4\pi} \int_{S'} \frac{\mathbf{M} \mathbf{J}_n(\mathbf{r}')}{\|\mathbf{r} - \mathbf{r}'\|^3} dS' \\ &\quad - \frac{3\mu_0}{4\pi} \int_{S'} \frac{((\mathbf{r} - \mathbf{r}')^T \mathbf{M} \mathbf{J}_n(\mathbf{r}'))(\mathbf{r} - \mathbf{r}')}{\|\mathbf{r} - \mathbf{r}'\|^5} dS' \end{aligned} \quad (24)$$

$$\mathbf{A}_n(\mathbf{r}) = \frac{\mu_0}{4\pi} \int_{S'} \frac{\mathbf{J}_n(\mathbf{r}')}{\|\mathbf{r} - \mathbf{r}'\|} dS' \quad (25)$$

$$W_{m,n} = \frac{\mu_0}{8\pi} \int_S \int_{S'} \frac{\mathbf{J}_m(\mathbf{r}) \cdot \mathbf{J}_n(\mathbf{r}')}{\|\mathbf{r} - \mathbf{r}'\|} dS' dS. \quad (26)$$

The $\mathbf{M} = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ is the last matrix of the 3D Levi-Civita tensor, which is useful to compactly express the gradient fields. Note that all the above integrals can be performed over the surface parameters using a change of variables:

$$\int_S f(\mathbf{r}) dS = \int_z \int_\theta f(\mathbf{r}(\theta, z)) \|\mathbf{n}(\theta, z)\| d\theta dz, \quad (27)$$

where $\mathbf{n}(\theta, z)$ is defined in equation (10). The final desired quantities can then be written as linear functions

of the stream basis coefficients

$$\mathbf{B}(\mathbf{r}) = \sum_n \mathbf{B}_n(\mathbf{r}) x_n \quad (28)$$

$$\frac{\partial B^{(z)}(\mathbf{r})}{\partial \mathbf{r}} = \sum_n \frac{\partial B_n^{(z)}(\mathbf{r})}{\partial \mathbf{r}} x_n \quad (29)$$

$$\mathbf{A}(\mathbf{r}) = \sum_n \mathbf{A}_n(\mathbf{r}) x_n \quad (30)$$

$$W = \sum_n \sum_m x_m W_{m,n} x_n. \quad (31)$$

Note that the inductance is defined as $L = \frac{1}{2} W I^2$, where I is the net current through the gradient coil. We typically assume unity current through the gradient coil, and hence the inductance is simply half the magnetic energy.

6.2.7 Improved Electric Field Calculations

Faraday's law tells us that an E-field is necessarily produced whenever there is a changing magnetic field

$$\nabla \times \mathbf{E} = -\frac{\partial}{\partial t} \mathbf{B}.$$

Since the magnetic field is divergence free $\nabla \cdot \mathbf{B} = 0$, we relate it to the curl of the magnetic vector potential

$$\mathbf{B} = \nabla \times \mathbf{A},$$

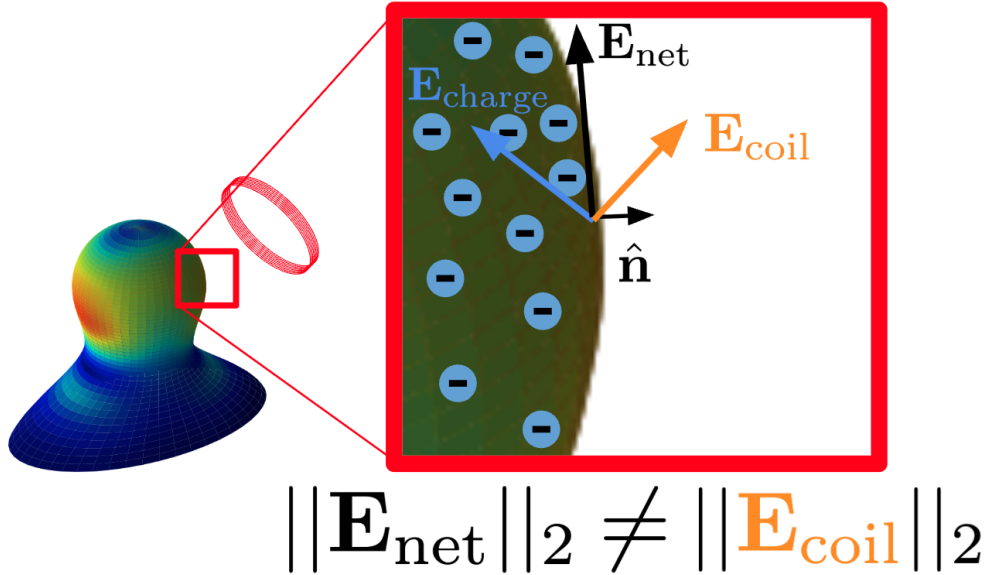


Figure 6: Illustration of physics corrected net electric field induced due to satisfaction of Gauss' Law

where $\mathbf{A}$ is chosen using the coulomb gauge. Combining the above two equations together yields

$$\nabla \times (\mathbf{E} + \frac{\partial}{\partial t} \mathbf{A}) = 0 \quad (32)$$

$$\implies \mathbf{E}_{\text{net}} = \underbrace{-\frac{\partial}{\partial t} \mathbf{A}}_{\mathbf{E}_{\text{coil}}} - \underbrace{\nabla \psi}_{\mathbf{E}_{\text{charge}}}, \quad (33)$$

where the second equation uses the fact that any curl-free field can be written as the gradient of a scalar function ψ . In a vacuum, it is valid to assume that $\nabla \psi = 0$, meaning that we can directly compute the electric field $\mathbf{E}$ from the magnetic vector potential $\mathbf{A}$. However, if our region of interest contains any conductive medium, we cannot ignore $\nabla \psi$. In such cases ψ is interpreted as an electrostatic potential due to charge accumulation.

The authors of [9, 10] propose using a boundary condition on the currents induced in the body to compute ψ , which can then be used to compute a more accurate electric field $\mathbf{E}$. The boundary condition comes from the fact that the imaging subject can be modeled as a conductive medium with conductivity σ , while the region just outside the imaging subject is non-conductive. Thus the current flowing out of the subject's surface, and hence the electric field (ohms law) must be zero:

$$\mathbf{J}_{\text{net}} \cdot \hat{\mathbf{n}}_{\text{pns}} = 0 \quad (34)$$

$$\implies (\sigma \mathbf{E}_{\text{net}}) \cdot \hat{\mathbf{n}}_{\text{pns}} = 0 \quad (35)$$

$\hat{\mathbf{n}}_{\text{pns}}$ is the normal vector at the body surface boundary. As suggested by the authors of [9, 10], we first need to model the surface charge density σ using a basis expansion

$$\sigma(u, v) = \sum_{m=1}^M \sum_{n=1}^N U_m(u) V_n(v) \alpha_{m,n} \quad (36)$$

The body surface is parameterized using $u \in [0, 2\pi]$ for angular position and $v \in [0, 1]$ for longitudinal position. The u bases $U_m(u)$ are sine and cosine functions, while the v bases $V_n(v)$ are triangular 'hat' functions (i.e. linear interpolators). The bases coefficients are denoted by $\alpha_{m,n}$. We assume access to some function mapping from the u, v coordinates to cartesian coordinates $\mathbf{r}(u, v) \in \mathbb{R}^3$.

The surface charge density distribution σ induces an electric field according to the differential form of Gauss's law

$$\nabla \cdot \mathbf{E}_{\text{charge}} = \frac{\sigma}{\epsilon_0}, \quad (37)$$

and the electric field is related to the electrostatic potential ψ according to

$$\mathbf{E}_{\text{charge}} = -\nabla \psi. \quad (38)$$

Combining the above expression yields Poisson's equation in ψ , which has a closed form integral solution shown below

$$\nabla^2 \psi = -\frac{\sigma}{\epsilon_0} \quad (39)$$

$$\implies \psi(\mathbf{r}) = \frac{1}{4\pi\epsilon_0} \int_S \frac{\sigma(\mathbf{r}')}{\|\mathbf{r} - \mathbf{r}'\|_2} dS', \quad (40)$$

and we can use (38) to compute the electric field from the charge density distribution

$$\mathbf{E}_{\text{charge}}(\mathbf{r}) = \frac{1}{4\pi\epsilon_0} \int_S \sigma(\mathbf{r}') \frac{\mathbf{r} - \mathbf{r}'}{\|\mathbf{r} - \mathbf{r}'\|_2^3} dS'. \quad (41)$$

The final step is to write $\mathbf{E}_{\text{charge}}$ using the u, v parameterization and the basis expansion in (36). The change of variables for integration is

$$\mathbf{n}_{\text{pns}}(u, v) = \frac{\partial \mathbf{r}(u, v)}{\partial u} \times \frac{\partial \mathbf{r}(u, v)}{\partial v} \quad (42)$$

$$\int_S f(\mathbf{r}) dS = \int_u \int_v f(\mathbf{r}(u, v)) \|\mathbf{n}_{\text{pns}}(u, v)\|_2 du dv. \quad (43)$$

Finally, we can write the electric field basis expansion as shown below

$$\mathbf{E}_{\text{charge}}(u, v) = \sum_{m=1}^M \sum_{n=1}^N \mathbf{E}_{m,n}(u, v) \alpha_{m,n} \quad (44)$$

$$\mathbf{E}_{m,n}(u, v) = \frac{1}{4\pi\epsilon_0} \int_u \int_v U_m(u) V_n(v) \frac{\mathbf{r}(u, v) - \mathbf{r}'(u, v)}{\|\mathbf{r}(u, v) - \mathbf{r}'(u, v)\|_2^3} \|\mathbf{n}_{\text{pns}}(u, v)\|_2 du dv \quad (45)$$

We can then combine equations (35), (33), and the electric-field basis expansion above to construct a linear system for the coefficients $\alpha_{m,n}$.

6.3 Fast Numerical Methods for Physics Integrals

Evaluating the integrals used to build the field matrices in Appendix 6.2.6 is the dominant cost in both the matrix-gradient and stream-function pipelines. The key observation from our computational implementation is that the optimization itself is comparatively cheap once the KPI matrices are available; the expensive step is repeatedly forming the columns of $\mathbf{G}$, $\mathbf{E}$, and $\mathbf{L}$ for every basis function or loop element and for every observation point of interest.

6.3.1 Quadrature Viewpoint

All of the continuous operators in (23)–(26) are ultimately reduced to weighted sums over source points on the conductor or source surface. Let $\{\mathbf{r}_j, \Delta S_j\}_{j=1}^{N_q}$ denote quadrature points and weights over a surface patch. Then a generic panel-based discretization takes the form

$$\mathbf{B}_n(\mathbf{r}_i) \approx \sum_{j=1}^{N_q} K_B(\mathbf{r}_i, \mathbf{r}_j) \mathbf{J}_n^j \Delta S_j, \quad (46)$$

$$\frac{\partial B_n^{(z)}(\mathbf{r}_i)}{\partial \mathbf{r}} \approx \sum_{j=1}^{N_q} K_G(\mathbf{r}_i, \mathbf{r}_j) \mathbf{J}_n^j \Delta S_j, \quad (47)$$

$$\mathbf{A}_n(\mathbf{r}_i) \approx \sum_{j=1}^{N_q} K_A(\mathbf{r}_i, \mathbf{r}_j) \mathbf{J}_n^j \Delta S_j, \quad (48)$$

where the kernels K_B , K_G , and K_A are induced by the Biot–Savart and vector-potential operators. For the inductance or magnetic-energy matrix, the corresponding discretization is a double sum,

$$W_{m,n} \approx \sum_{i=1}^{N_q} \sum_{j=1}^{N_q} K_L(\mathbf{r}_i, \mathbf{r}_j) (\mathbf{J}_m^i \cdot \mathbf{J}_n^j) \Delta S_i \Delta S_j. \quad (49)$$

This viewpoint is useful because it makes explicit which quantities can be precomputed and which quantities must be recomputed whenever the geometry parameters $\boldsymbol{\theta}$ change.

To make the numerical implementation explicit, we now write the integral operators in discrete quadrature form. This appendix applies to both the stream-function surface integrals and the matrix-coil loop integrals. The continuous operators being discretized are the magnetic-field, gradient-field, vector-potential, and magnetic-energy integrals together with the surface-charge electric-field integrals.

Generic quadrature rule. For an integral over a parameter domain Ξ ,

$$\int_{\Xi} f(\xi) d\xi \approx \sum_{q=1}^{N_q} w_q f(\xi_q),$$

where ξ_q are quadrature nodes and w_q are the corresponding weights. For tensor-product surface quadrature over (θ, z) , we use

$$\int \int f(\theta, z) d\theta dz \approx \sum_{i=1}^{N_\theta} \sum_{j=1}^{N_z} w_i^{(\theta)} w_j^{(z)} f(\theta_i, z_j).$$

Midpoint / panel quadrature. If the parameter interval $[a, b]$ is partitioned into N_p panels of width $\Delta\xi = (b - a)/N_p$, then midpoint quadrature gives

$$\int_a^b f(\xi) d\xi \approx \sum_{p=1}^{N_p} \Delta\xi f\left(a + \left(p - \frac{1}{2}\right) \Delta\xi\right).$$

For a surface parameterization over (θ, z) , this becomes

$$\int \int f(\theta, z) d\theta dz \approx \sum_{i=1}^{N_\theta} \sum_{j=1}^{N_z} \Delta\theta \Delta z f(\theta_i^{\text{mid}}, z_j^{\text{mid}}),$$

with midpoint samples

$$\theta_i^{\text{mid}} = \theta_{\min} + \left(i - \frac{1}{2}\right) \Delta\theta, \quad z_j^{\text{mid}} = z_{\min} + \left(j - \frac{1}{2}\right) \Delta z.$$

Gauss–Legendre quadrature. For a 1D integral on $[-1, 1]$, Gauss–Legendre quadrature gives

$$\int_{-1}^1 f(x) dx \approx \sum_{q=1}^{N_q} w_q^{\text{GL}} f(x_q^{\text{GL}}),$$

where x_q^{GL} and w_q^{GL} are the standard Gauss–Legendre nodes and weight [11]. For a general interval $[a, b]$, the affine change of variables

$$\xi = \frac{b-a}{2}x + \frac{a+b}{2}$$

yields

$$\int_a^b f(\xi) d\xi \approx \frac{b-a}{2} \sum_{q=1}^{N_q} w_q^{\text{GL}} f\left(\frac{b-a}{2}x_q^{\text{GL}} + \frac{a+b}{2}\right).$$

Discrete stream-function magnetic field. Using the surface change of variables $dS = \|n(\theta, z)\| d\theta dz$, the basis-field integral is discretized as

$$B_n(r) \approx \frac{\mu_0}{4\pi} \sum_{i=1}^{N_\theta} \sum_{j=1}^{N_z} w_i^{(\theta)} w_j^{(z)} \frac{J_n(r(\theta_i, z_j)) \times (r - r(\theta_i, z_j))}{\|r - r(\theta_i, z_j)\|^3} \|n(\theta_i, z_j)\|.$$

Similarly, the vector-potential basis term becomes

$$A_n(r) \approx \frac{\mu_0}{4\pi} \sum_{i=1}^{N_\theta} \sum_{j=1}^{N_z} w_i^{(\theta)} w_j^{(z)} \frac{J_n(r(\theta_i, z_j))}{\|r - r(\theta_i, z_j)\|} \|n(\theta_i, z_j)\|.$$

Discrete magnetic-energy matrix. The double-surface integral for $W_{m,n}$ becomes

$$W_{m,n} \approx \frac{\mu_0}{8\pi} \sum_{i=1}^{N_\theta} \sum_{j=1}^{N_z} \sum_{k=1}^{N_\theta} \sum_{l=1}^{N_z} w_i^{(\theta)} w_j^{(z)} w_k^{(\theta)} w_l^{(z)} \frac{J_m(r_{ij}) \cdot J_n(r_{kl})}{\|r_{ij} - r_{kl}\|} \|n_{ij}\| \|n_{kl}\|,$$

where $r_{ij} = r(\theta_i, z_j)$ and $n_{ij} = n(\theta_i, z_j)$.

Discrete charge-induced electric field. For the electric-field basis functions in (45), the discrete surface quadrature is

$$E_{m,n}(u, v) \approx \frac{1}{4\pi\epsilon_0} \sum_{i=1}^{N_u} \sum_{j=1}^{N_v} w_i^{(u)} w_j^{(v)} U_m(u_i) V_n(v_j) \frac{r(u, v) - r(u_i, v_j)}{\|r(u, v) - r(u_i, v_j)\|^3} \|n_{\text{pns}}(u_i, v_j)\|.$$

Elliptical loop formulation. For a single elliptical loop parameterized by

$$r(\phi) = \begin{bmatrix} a \cos \phi \\ b \sin \phi \\ z_0 \end{bmatrix}, \quad \phi \in [0, 2\pi],$$

the line integral

$$\int_0^{2\pi} g(\phi) d\phi$$

may be approximated either by midpoint quadrature,

$$\int_0^{2\pi} g(\phi) d\phi \approx \sum_{p=1}^{N_p} \Delta\phi g\left(\left(p - \frac{1}{2}\right) \Delta\phi\right), \quad \Delta\phi = \frac{2\pi}{N_p},$$

or by Gauss–Legendre quadrature,

$$\int_0^{2\pi} g(\phi) d\phi \approx \pi \sum_{q=1}^{N_q} w_q^{\text{GL}} g(\pi x_q^{\text{GL}} + \pi).$$

6.3.2 Where the Compute Goes

Let N denote the number of design coefficients, $N_G = |\Omega_{\text{dsv}}|$ the number of gradient evaluation points, $N_E = |\Omega_{\text{pns}}|$ the number of electric-field evaluation points, and N_q the number of source quadrature points per basis function. The KPI matrices are then assembled as

$$\mathbf{G} \in \mathbb{R}^{N_G \times N}, \quad G_{i,n} = G_n(\mathbf{r}_i), \forall \mathbf{r}_i \in \Omega_{\text{dsv}} \quad (50)$$

$$\mathbf{E} \in \mathbb{R}^{N_E \times N}, \quad E_{j,n} = E_n(\mathbf{r}_j), \forall \mathbf{r}_j \in \Omega_{\text{pns}} \quad (51)$$

$$\mathbf{L} \in \mathbb{R}^{N \times N}. \quad (52)$$

The dominant scaling is summarized in Table 1. The exact wall-clock time depends strongly on caching, symmetry reuse, and GPU implementation details, but the table captures which terms dominate asymptotically.

In our representative setup, the stream-function approach uses roughly $N \approx 40$ basis coefficients, around $N_G \approx 4 \times 10^3$ DSV points, and around $N_E \approx 2.5 \times 10^3$ PNS points. Consequently, building $\mathbf{G}$ and $\mathbf{E}$ already requires tens of thousands of vector–vector interactions per basis function, while the matrix-gradient

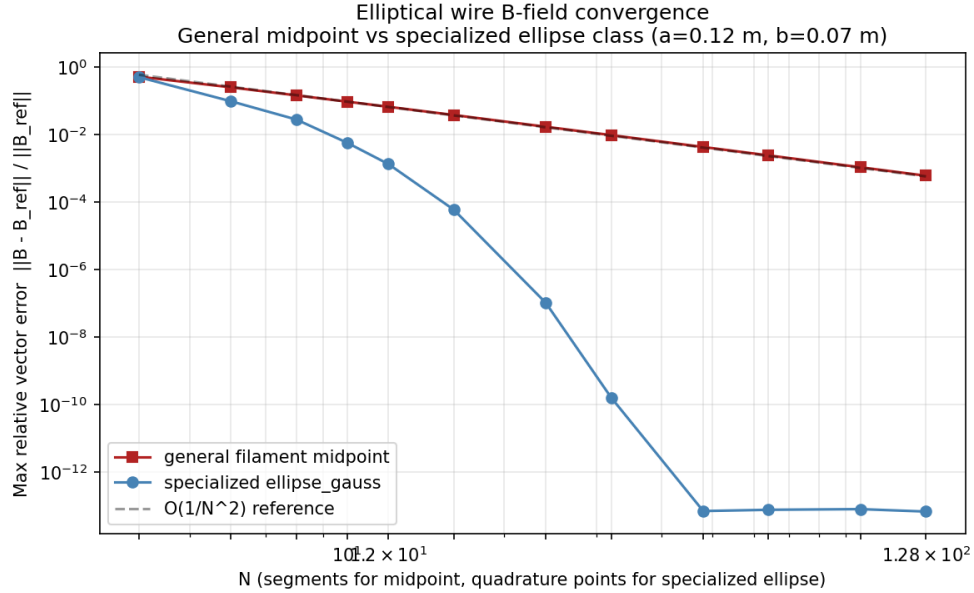


Figure 7: Comparative convergence of discrete magnetic field computations using midpoint and gauss-legendre quadrature.

Table 1: Representative computational cost of the main pipeline stages. Here N_b is the number of basis functions (or stream-function coefficients), N_G is the number of DSV evaluation points, N_E is the number of PNS evaluation points, and q_n is the number of quadrature nodes used for basis function n . When a uniform quadrature rule is used, we write $q_n = \bar{q}$ for all n .

Stage	Matrix-gradient formulation	Stream-function / filament formulation	Notes
Form one column $\mathbf{G}_{:,n}$	$\mathcal{O}(N_G q_n)$	$\mathcal{O}(N_G q_n)$	One Biot–Savart evaluation from basis element n to all DSV points.
Form one column $\mathbf{E}_{:,n}$	$\mathcal{O}(N_E q_n)$	$\mathcal{O}(N_E q_n)$	Same structure as $\mathbf{G}$, but evaluated on the PNS/body surface.
Form one entry L_{mn}	$\mathcal{O}(q_m q_n)$	$\mathcal{O}(q_m q_n)$	Double quadrature over interacting basis elements m, n .
Assemble full $\mathbf{G} \in \mathbb{R}^{N_G \times N_b}$	$\mathcal{O}\left(N_G \sum_{n=1}^{N_b} q_n\right)$	$\mathcal{O}\left(N_G \sum_{n=1}^{N_b} q_n\right)$	If $q_n = \bar{q}$, this becomes $\mathcal{O}(N_G N_b \bar{q})$. Parallel over target points and basis index.
Assemble full $\mathbf{E} \in \mathbb{R}^{N_E \times N_b}$	$\mathcal{O}\left(N_E \sum_{n=1}^{N_b} q_n\right)$	$\mathcal{O}\left(N_E \sum_{n=1}^{N_b} q_n\right)$	If $q_n = \bar{q}$, this becomes $\mathcal{O}(N_E N_b \bar{q})$. Often similar kernel structure to $\mathbf{G}$.
Assemble full $\mathbf{L} \in \mathbb{R}^{N_b \times N_b}$	$\mathcal{O}\left(\sum_{m=1}^{N_b} \sum_{n=1}^{N_b} q_m q_n\right)$	$\mathcal{O}\left(\sum_{m=1}^{N_b} \sum_{n=1}^{N_b} q_m q_n\right)$	If $q_n = \bar{q}$, this becomes $\mathcal{O}(N_b^2 \bar{q}^2)$. Usually the most expensive dense assembly step.
One ADMM $\mathbf{x}$ -update	Dense direct solve: $\mathcal{O}(N_b^3)$ after matrix assembly; iterative solve: problem-dependent, typically matrix-vector dominated		
Full ADMM solve (K iterations)	$\mathcal{O}(N_b^3 + K N_b^2)$ for a prefactored dense system, or approximately K times the iterative solve cost		If matrices are cached, this is often cheaper than rebuilding $\mathbf{G}, \mathbf{E}, \mathbf{L}$.
One outer geometry iteration	Recompute geometry dependent operators $\mathbf{G}_{\text{surf}}, \mathbf{E}_{\text{surf}}, \mathbf{L}_{\text{surf}}$, then backpropagate through the unrolled solver		

approach with elliptical loops can often use on the order of 128 quadrature points per loop. This is exactly why a specialized quadrature rule and aggressive parallelization matter.

6.3.3 Implementation Notes: Parallelization and Kernel Fusion

The integral kernels are embarrassingly parallel over basis index, observation point, and quadrature point. For example, when forming an inductance entry $L_{m,n}$, each block of threads can be assigned a fixed pair (m, n) , while individual threads accumulate contributions from subsets of quadrature pairs (i, j) before

performing a block-level reduction.

A second practical lesson is that the formations of $\mathbf{G}$ and $\mathbf{E}$ share substantial geometric work: source-point positions, tangent vectors, distances, and inverse-distance powers are all reused. When those intermediate quantities are not needed elsewhere, it is advantageous to fuse the kernels so the shared intermediates are computed once and immediately reduced into both outputs. This is yet to be in our implementation, but one can see this lowering memory traffic and improves GPU throughput.

6.4 ADMM Updates

Below are the ADMM updates to solve (1):

$\mathbf{x}$ Updates

$$\begin{aligned}\mathbf{A}^T \mathbf{A}^{(t)} &= \rho_G \mathbf{G}^T \mathbf{G} + \rho_L \mathbf{L}^T \mathbf{L} + \rho_E \sum_k \mathbf{E}_k^T \mathbf{E}_k \\ \mathbf{A}^T \mathbf{b}^{(t)} &= \rho_G \mathbf{G}^T (\mathbf{s}_G^{(t)} - \mathbf{d}_G^{(t)}) + \rho_L \mathbf{L}^T (\mathbf{s}_L^{(t)} - \mathbf{d}_L^{(t)}) + \rho_E \sum_k \mathbf{E}_k^T (\mathbf{s}_k^{(t)} - \mathbf{d}_k^{(t)}) \\ \mathbf{x}^{(t+1)} &\leftarrow \text{solve}(\mathbf{A}^T \mathbf{A}^{(t)}, \mathbf{A}^T \mathbf{b}^{(t)})\end{aligned}\tag{53}$$

$G_{\min}$ Updates

First we define $\mathbf{q}_G := \mathbf{G}\mathbf{x}^{(t+1)} + \mathbf{d}_G^{(t)}$, and then solve

$$\begin{aligned}\mathbf{s}_G^{(t+1)}, G_{\min}^{(t+1)} &\leftarrow \arg \min_{\mathbf{s}, G} -\lambda_G G + \frac{\rho_G}{2} \|\mathbf{s} - \mathbf{q}_G\|_2^2 \\ \text{s.t. } \mathbf{s} &\geq G.\end{aligned}\tag{54}$$

$L_{\max}$ Updates

First we define $\mathbf{q}_L := \mathbf{L}\mathbf{x}^{(t+1)} + \mathbf{d}_L^{(t)}$, and the updates are

$$\begin{aligned}\mathbf{s}_L^{(t+1)}, L_{\max}^{(t+1)} &\leftarrow \arg \min_{\mathbf{s}, L} -\lambda_L L + \frac{\rho_L}{2} \|\mathbf{s} - \mathbf{q}_L\|_2^2 \\ \text{s.t. } \|\mathbf{s}\|_2^2 &\leq L.\end{aligned}\tag{55}$$

This optimization has the following closed-form updates:

$$\begin{aligned}\mathbf{s}_L^{(t+1)} &\leftarrow \mathbf{q}_L \frac{\rho_L}{\rho_L + 2\lambda_L} \\ L_{\max}^{(t+1)} &\leftarrow \|\mathbf{s}_L^{(t+1)}\|_2^2\end{aligned}\tag{56}$$

$E_{\max}$ Updates

First we define $\mathbf{q}_k := \mathbf{E}_k \mathbf{x}^{(t+1)} + \mathbf{d}_k^{(t)}$, $\forall k$ and then solve

$$\begin{aligned}\mathbf{s}_1^{(t+1)}, \dots, \mathbf{s}_K^{(t+1)}, E_{\max}^{(t+1)} &\leftarrow \arg \min_{\mathbf{s}_1, \dots, \mathbf{s}_K, E} -\lambda_E E + \frac{\rho_E}{2} \sum_k \|\mathbf{s}_k - \mathbf{q}_k\|_2^2 \\ \text{s.t. } \|\mathbf{s}_k\|_2 &\leq E, \forall k\end{aligned}\tag{57}$$

C Updates

First we define $\mathbf{q}_C := \mathbf{C}\mathbf{x}^{(t+1)} + \mathbf{d}_C^{(t)}$, and then solve

$$\begin{aligned} \mathbf{s}_C^{(t+1)} \leftarrow \arg \min_{\mathbf{s}} \|\mathbf{s} - \mathbf{q}_C\|_2^2 \\ \text{s.t. } |\mathbf{s}| \leq d. \end{aligned} \tag{58}$$

This optimization has the following closed-form update:

$$s_C^{(t+1)}[i] \leftarrow \text{clip}(q_C[i], -d, d), \forall i \tag{59}$$